

Lesson 11 — Saving And Loading

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Four facts

1. **strings only** — `getItem` always returns a string or `null`
2. **per origin** — `file://` and `localhost:8000` have separate storage
3. **about 5 MB**
4. **it can THROW** — in a private window, even reading

The whole storage layer

```
const Store = {
  available: false,
  init() {
    try {
      localStorage.setItem("__t", "1");
      localStorage.removeItem("__t");
      this.available = true;
    } catch (e) { this.available = false; }
  },
  load(key, fallback) {
    if (!this.available) return fallback;
    try {
      const t = localStorage.getItem(key);
      if (t === null) return fallback;
      return JSON.parse(t);
    } catch (e) { return fallback; }
  },
  save(key, value) {
    if (!this.available) return false;
    try {
      localStorage.setItem(key,
        JSON.stringify(value));
      return true;
    } catch (e) { return false; }
  }
};
```

Every load takes a **fallback**, so "no save" is never a special case.

JSON is lossy and silent

in	out
number, string, boolean	same
array, plain object	same
function	key gone
<code>undefined</code>	key gone
<code>Infinity</code> , <code>NaN</code>	<code>null</code>
<code>Date</code>	a string
<code>Map</code> , <code>Set</code>	<code>{}</code>
circular	throws

Save only numbers, strings, booleans, arrays, plain objects.

Loaded data is untrusted input

```
return {
  volume: clampNumber(raw.volume, 0, 1,
    0.3),
  shake: typeof raw.shake === "boolean"
    ? raw.shake : true
};
```

Build a **clean** object. Check the type *before* clamping — `Math.min(1, "banana")` is `NaN` and sails straight through.

Version from day one

```
if (save.version === 1) { /* 1 → 2 */
  save.version = 2; }
if (save.version === 2) { /* 2 → 3 */
  save.version = 3; }
if (save.version !== SAVE_VERSION) return
null; // from the future
```

One step at a time. Treat a missing `version` as 1.

When

On events, plus:

```
document.addEventListener("visibilitychange"
, () => {
    if (document.visibilityState === "hidden")
saveNow();
});
```

Never every frame.

Do not save

derived values · references to objects · the whole
game object

Section A — Recall

A1. Give the four facts about `localStorage`, and say which one would stop a game from starting at all. [4 marks]

A2. Name three kinds of value that do not survive `JSON.stringify`, and say what each becomes. [3 marks]

A3. Give three completely different reasons the data you load might not be what you saved. [3 marks]

A4. Why does `Store.load` take a *fallback* argument? [2 marks]

A5. Why migrate one version at a time instead of writing one function that handles every old format? [3 marks]
What should happen to a save from a *newer* version?

A6. Why should you not save the number of coins remaining in a level? [2 marks]

A7. Why not save on every frame, and when should you save instead? [2 marks]

Section B — Predict the output

B1. What is printed, and why? [3 marks]

```
localStorage.setItem("score", 4200);  
console.log(localStorage.getItem("score") + 1);
```

B2. What does `loaded` contain? Be precise about every key.

[4 marks]

```
const save = { hp: 100, best: Infinity, when: new Date(),
               seen: new Set([1, 2]), heal: function () {} };
const loaded = JSON.parse(JSON.stringify(save));
```

B3. There is no save yet. What does this produce, and when does the problem actually show up?

[3 marks]

```
const save = JSON.parse(localStorage.getItem("save"));
console.log(save.level);
```

B4. The player has edited the save to `{"volume": "loud"}`. What does `volume` end up as?

[3 marks]

```
const volume = Math.max(0, Math.min(1, raw.volume));
```

B5. A save has `version: 1` and the chain handles `1→2` and `2→3`. Walk through what happens. Now the save has `version: 5` and `SAVE_VERSION` is 3 — what happens then?

[3 marks]

Section C — Find and fix the bug

C1. The game works perfectly for the author and does not start at all for one of their friends. No error is ever reported to the author.

```
const settings = JSON.parse(localStorage.getItem("settings"));
startGame(settings);
```

C2. After loading a save, `TypeError: player.dash is not a function` appears — but only in games that were loaded, never in new ones. [3 marks]

```
const player = { x: 0, dash: function () { /* ... */ } };
Store.save("player", player);
// next session:
const player = Store.load("player", {});
```

C3. The high score table shows 9, 850, 92, 1000 in that order. [3 marks]

```
scores.sort();
```

C4. The best time is displayed as "null" after the first game, and every later comparison against it fails. [4 marks]

```
let best = Infinity;
if (time < best) { best = time; }
Store.save("best", best); // on the very first run, before any game
```

C5. The game stutters about once a second, and the frame-time graph shows a spike each time. [3 marks]

```
function update(dt) {
  movePlayer(dt);
  Store.save("save", makeSave(game));
}
```

Section D — Build it

- ☐ **Checkpoint 1.** Write the `Store` wrapper with `init`, `load` and `save`. Show `Store.available` on screen. Then open your game in a private window and check it still runs.
- ☐ **Checkpoint 2.** Save and restore one number — a high score. Prove the string problem to yourself by adding 1 to it before you convert it.
- ☐ **Checkpoint 3.** Save a settings object: volume, a toggle, and a control scheme. Change them, reload, confirm they stick.
- ☐ **Checkpoint 4.** Write a validator for those settings that builds a clean object from defaults. Then edit the save in devtools to something absurd and confirm the game still starts with sensible values.
- ☐ **Checkpoint 5.** Build a top-ten high score table with names. Sort it correctly, keep ten, and validate every entry on load (a name must be a string; a score must be a finite number).

- ☐ **Checkpoint 6.** Add a `version` to your save and a migration chain. Then hand-write an old-format save into storage and watch it upgrade.
- ☐ **Checkpoint 7.** Save on events plus `visibilitychange`, and show on screen when the last save happened. Confirm you are not saving every frame.
- ☐ **Checkpoint 8.** Add a “corrupt my save” button that damages it five different ways: truncated text, wrong types, missing fields, a future version, and not JSON at all. All five must load without crashing.

THE TEST THAT MATTERS

Open your game in a private browsing window. If it does not start, you have the

bug this whole lesson exists to prevent — and it is the one bug you will never find by playing your own game normally.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. A high score table stores a name the player typed. List five things somebody might put in that box that you would not want in your table, and say what you would do about each.

E2. A player edits their save so `level: 99`, which does not exist. What should the game do? Now they edit `score: 999999`. Same kind of problem? Same answer?

E3. Design three save slots, each showing level, score and when it was last played. What do you store, and under what keys? What happens when a slot is overwritten — do you ask first?

E4. The hard one. Instead of saving every tile of a generated dungeon, save the **seed** the generator started from and rebuild it. A few bytes instead of a megabyte. What has to be true of your generator for that to work? What breaks the moment you change the generator?

Stretch goals

1. Export and import the save as text the player can copy. (Also a cheating device — worth discussing.)
 2. A checksum that refuses an edited save. Honest about what it does and does not stop.
 3. Three save slots (E3).
 4. A settings scene pushed onto your lesson 7 stack, writing on every change.
 5. The five-way corruption button, and surviving all of it.
-